BTS CIEL · PROGRAMMATION C++ · EXERCICES

Exercices — Chapitre 6 : Représentation des données et opérations bit à bit

Cours associé : `C06_Representation_donnees_bit_a_bit` · **Fichiers de départ :** `02_Exercices/fichiers/E06/`

> **Méthode :** pour chaque exercice, faites **d'abord** le calcul à la main (binaire posé sur papier), **puis** vérifiez avec un programme. Le calcul à la main est attendu à l'évaluation.

N°	Titre	Niveau	Notions du cours
6.1	Conversions et opérations	1 — Application	§1.1, §2.1, §2.3
6.2	Prévoir l'affichage	1 — Application	§1.1, §1.3, §2.1, §2.2
6.3	XOR à la main	1 — Application	§2.1, §2.4
6.4	Gestion de drapeaux	2 — Entraînement	§3.1–§3.3
6.5	Décoder une trame textuelle	2 — Entraînement	§1.2–§1.3 ; découpage de champs précisé dans l'énoncé
6.6	Extraire les octets d'une adresse	2 — Entraînement	§1.2, §2.3, §4.2
6.7	Calculer une adresse réseau	3 — Problème	§2.1, §4.1–§4.2

Niveau 1 — Application directe

Exercice 6.1 — Conversions et opérations

Objectif : passer d'une base à l'autre et appliquer les opérateurs bit à bit.

Énoncé :

1. Complétez le tableau :

Décimal	Binaire (8 bits)	Hexadécimal
10		
	0011 0000	
		0x7F
192		

2. Calculez à la main : `0b0101 & 0b0011`, `0b1100 | 0b1010`, `0b1111 ^ 0b0101`.
3. Calculez à la main : `5u << 1`, `5u << 2`, `20u >> 1`, `20u >> 2`. Quelle règle relie ces résultats aux multiplications et divisions par 2 ?
4. Vérifiez toutes vos réponses avec un programme C++.

Exercice 6.2 — Prévoir l'affichage

Objectif : relier extraction de quartets, XOR, affichage hexadécimal et codes ASCII.

Énoncé : sans exécuter le programme `ex6_2.cpp`, indiquez les trois lignes qu'il affiche, en détaillant chaque calcul en binaire. Vérifiez ensuite en compilant.

```cpp
#include <iostream>

int main() {
    unsigned int v{0xA5};

    std::cout << ((v >> 4) & 0xF) << ' ' << (v & 0xF) << '\n';
    std::cout << std::hex << (v ^ 0xFF) << std::dec << '\n';
    std::cout << static_cast<char>(0x41 + 2) << '\n';

    return 0;
}
```

Exercice 6.3 — XOR à la main

Objectif : appliquer le XOR avec une clé répétée.

Énoncé : on applique la propriété du XOR avec une clé répétée (§2.4) : clé `10 20 30 40`, données masquées `53 69 75 0C 31` (en hexadécimal).

1. Démasquez à la main le **premier** octet : écrivez-le en binaire, appliquez le XOR avec l'octet de clé, convertissez en hexadécimal, puis en caractère.
2. Faites de même pour le **dernier** octet (indice 4). Quel octet de clé utilise-t-il ? Pourquoi ?

Résultat attendu : les caractères `C` et `!`, avec le détail des calculs.

Niveau 2 — Entraînement

Exercice 6.4 — Gestion de drapeaux

Objectif : tester, activer, désactiver des bits avec des masques.

Énoncé : l'état d'un équipement est codé sur 4 bits :

Bit	Signification
0	Alimentation
1	Capteur connecté
2	Alarme
3	Maintenance

Partant de l'état `0001`, effectuez dans l'ordre : activer le capteur, tester l'alarme (afficher le résultat), activer l'alarme, désactiver l'alimentation. Affichez l'état final en décimal.

Contraintes : une constante `constexpr` par bit ; calculez d'abord chaque état intermédiaire à la main.

Résultat attendu : les quatre états intermédiaires en binaire, le résultat du test d'alarme et l'état final en décimal.

Exercice 6.5 — Décoder une trame textuelle

Objectif : convertir des octets en caractères ASCII et isoler des champs.

Énoncé : un analyseur relève la trame `0x33 0x2F 0x34 0x31 0x2F 0x37`. Chaque octet représente un caractère ASCII ; le caractère `/` sépare les trois champs. Les champs sont des chaînes de caractères, pas nécessairement des nombres de longueur fixe.

1. Convertissez chaque octet en décimal, puis en caractère ASCII.
2. La trame suit le format `adresse/commande/donnee` : donnez la valeur de chaque champ.
3. Écrivez le programme de décodage : il affiche la trame en texte, puis chaque champ.

Exercice 6.6 — Extraire les octets d'une adresse

Objectif : extraire des octets d'un entier de 32 bits.

Énoncé : en suivant la convention du cours (octet 1 dans les bits 31 à 24), l'adresse `192.168.1.10` est stockée dans un `std::uint32_t` initialisé à `0xC0A8010Au`. Extrayez chaque octet avec `>>` et `& 0xFF`, puis affichez l'adresse en notation pointée.

Résultat attendu :

```
192.168.1.10
```

Niveau 3 — Résolution de problèmes

Exercice 6.7 — Calculer une adresse réseau

Objectif : appliquer un masque par ET bit à bit.

Énoncé : les préfixes indiquent le nombre de bits à 1 dans le masque.

1. Pour `192.168.1.77/26` (masque `255.255.255.192`), convertissez 77 et 192 en binaire, calculez `77 & 192` et donnez l'adresse réseau. Vérifiez en C++.
2. Calculez l'adresse réseau de `192.168.1.230` avec `/24`, `/25`, `/26` et `/27`. Expliquez pourquoi elle change avec le préfixe.